ST0527 · SECURE CODING

# Visual Screening Management System

Secure, offline-capable community screening operations

Nachiketh Reddy · Keefe Chen Lin Li · Mike Franco Abat · Sining (Sitt)

13 August 2026

| IMPLEMENTED | MEASURED | OPEN PROOF |
| --- | --- | --- |
| 4 stations + PWA | 500 participants + restore | Current AWS acceptance replay |

# Report map

This report follows the required documentation sections and keeps implementation, dated deployment evidence, repeatable test evidence and remaining human-owned proof distinct.

Repository-side report source. Implementation claims link to source or dated evidence. The 11 August deployment record is historical; the final acceptance replay and human-owned submission fields remain explicitly identified.

# 1. Executive summary

VSMS replaces paper hand-offs at community vision-screening events with an auditable workflow. A React/Vite progressive web application is backed by a Node.js Express REST API and Prisma-managed PostgreSQL. Staff can create and publish staffed events, register consenting participants, issue QR passes, operate virtual queues, record all four required screening types (visual acuity, refraction, colour vision and eye health), synchronize encrypted offline captures, complete Doctor review, issue referrals, and view or export aggregate reports.

The security model treats authentication and authorization as separate decisions. Amazon Cognito performs authorization-code-with-PKCE sign-in; the API then intersects the authenticated identity with active local account state, event membership, role and station duty before allowing protected operations. Zod validation, CSRF protection, rate limits, idempotency, transactional writes, encrypted owner-scoped IndexedDB and immutable audit records reduce risk around NRIC and clinical data.

The application was deployed on 11 August 2026 using AWS Amplify managed hosting, an Nginx/Express EC2 host, private encrypted RDS PostgreSQL and Cognito. That deployment is recorded in docs/2026-08-11 aws-cloud-deployment-runbook.md. During the 13 August audit, the Amplify frontend remained reachable but the API health endpoint timed out; therefore this report does not misrepresent historical deployment proof as a current end-to-end acceptance pass. A reproducible acceptance matrix and demo run sheet are ready for replay as soon as authorized AWS access is renewed.

# 2. Scope and evidence rules

The canonical implementation sources are:

| Area | Evidence | What it supports |
|---|---|---|
| Browser | react-user-dashboard/src/features/screening/offlineSync.ts, OfflineSyncProvider.tsx, station pages, Vite PWA config | Installable React/Vite PWA, scoped encrypted IndexedDB pack, four-station sync and conflict handling |
| API | backend/app.js, backend/server.js, backend/routes/, backend/controllers/, backend/services/ | Express request path, middleware, service and worker boundaries |
| Contract | backend/docs/openapi.yaml, backend/scripts/check-contract.js | Versioned API paths, operation IDs, response/security contract |
| Database | backend/prisma/schema.prisma, backend/prisma/migrations/ | PostgreSQL provider, relational models, constraints, indexes and migration-owned objects |
| SQL evidence | backend/stored procedures.sql | A separate stored-procedure/function draft; not proof that those objects are installed or called by the current Prisma path |
| Auth/config | backend/config/env.js, backend/utils/cognitoClient.js, infrastructure/cognito.yaml | Cognito integration/configuration boundary and fail-closed environment validation |
| Logging | backend/utils/logger/logger.js, backend/middlewares/httpLogger.js, backend/app.js, and backend/tests/unit/http-logging.test.js | Pino redaction and correlated HTTP completion logging |

| Area | Evidence | What it supports |
|---|---|---|
| Tests | backend/tests/, react-user-dashboard/src//*.test.*, .github/workflows/ | Runnable checks, 500-participant load workflow and backup/restore verification; not a substitute for live acceptance evidence |

docs/vsms-client-brief.md describes project requirements and alternatives. Reference material is not treated as proof that an alternative architecture was implemented.

The combined repository uses Pino for structured logging, pino-http for correlated HTTP completion records, and the request boundary versioned route and middleware → controller → service → Prisma Client → PostgreSQL. Controllers map HTTP input/output; services own domain decisions, authorization-sensitive checks, Prisma access and transactions. There is no repository layer; backend/docs/request-architecture.md records the same boundary.

# 3. Implemented architecture

```
Browser React/Vite dashboard
  ├─ authenticated API client and role-aware routes
  ├─ station forms and queue/review/report screens
  └─ encrypted IndexedDB screening pack and sync outbox
        │ HTTPS / same-origin proxy or configured API origin
        ▼
Node.js + Express API
  ├─ auth, CSRF, rate limits, request context and validation middleware
  ├─ route → controller → service modules
  ├─ separate Node worker processes launched from `backend/scripts/`
  └─ Prisma client
        │ DATABASE_URL
        ▼
PostgreSQL
  ├─ relational constraints, indexes and transactions
  ├─ append-only audit and sync transition records
  └─ event, registration, queue, screening, review and referral data

External identity boundary: Cognito authorization-code + PKCE flow when
configured; the API still owns local account state and event authorization.
```

## Runtime and deployment boundary

• Express: backend/app.js mounts the API and middleware; backend/server.js

owns startup, TLS-aware local transport and graceful shutdown. Worker scripts under backend/scripts/ are separate Node processes operated alongside the Express process; they are not in-process Express workers.

• AWS deployment: the dated runbook records Amplify hosting and a

same-origin /api/* proxy to Nginx with Let's Encrypt on a t3.small EC2 instance. The API and standalone workers run under systemd. This is a lab-grade Single-AZ topology, not a high-availability production claim.

• PostgreSQL: Prisma declares provider = "postgresql"; migrations are

the runtime schema authority. backend/db/init.sql intentionally refuses to create the old incompatible schema.

• Logging: backend/utils/logger/logger.js uses Pino

with redaction, and backend/middlewares/httpLogger.js uses pino-http for correlated HTTP completion records. backend/app.js mounts it after request context, and backend/tests/unit/http-logging.test.js checks redaction, correlation and completion fields.

• Identity: the dated runbook records a deployed Cognito user pool, PKCE

callback and administrator alignment. Current provider health still belongs in the final live replay.

### Implemented, planned and deferred services

| Status | Service/capability | Evidence and boundary |
|---|---|---|
| Implemented | Express REST API | backend/app.js, backend/routes/, OpenAPI contract |
| Implemented | PostgreSQL/Prisma persistence | backend/prisma/schema.prisma, migrations, services |
| Implemented | Event, membership, station and queue operations | backend/services/event/, backend/services/screening/, queue routes |
| Implemented | Participant, consent, QR and registration flows | backend/services/participant/, participant/registration/QR routes |
| Implemented | Visual acuity, refraction, colour-vision and eye-health save paths | screeningService.js, corresponding OpenAPI operations, frontend station pages and offline sync schemas |
| Implemented | Review, referral and aggregate report/export source paths | reviewService.js, referralService.js, services/reporting/ |
| Implemented | Transactional outbox and separate Node worker processes | domainEventBus.js, scripts/domain-event-worker.js, scripts/report-worker.js, and scripts/lifecycle-email-worker.js; each is a standalone entry point over shared PostgreSQL state |
| Implemented | Installable PWA and scoped offline screening pack | backend/docs/offline-screening-28.md, offlineSync.ts, Vite PWA configuration and generated manifest.json |
| Config-dependent | Cognito staff identity and provider synchronization | Cognito client/config and provider-operation services; environment/provider evidence required |
| Config-dependent | OneMap, SES/SNS and Redis integrations | Provider adapters and environment settings exist; external delivery/availability is not claimed |
| Implemented | Pino/Pino HTTP structured and correlated logging | backend/utils/logger/logger.js, backend/middlewares/httpLogger.js, backend/app.js, and the HTTP logging tests |
| Deferred | Participant self-service offline, full sync-centre UI and broader offline coverage | Explicitly out of scope in the offline implementation note |

# 4. Requirements and API map

The requirement-to-API inclusion point is api-requirement-map.md. It maps each core requirement to the actual OpenAPI operation IDs, route/controller/service sources, and current status. The map intentionally uses the current auth and sync routes rather than the older illustrative /auth/login and /sync/batch paths.

The core mapping is:

| Requirement | Current API evidence | Status |
|---|---|---|
| FR-01 Event management | Event list/create/detail/update/delete, lifecycle commands, stations, shifts and memberships | Implemented in OpenAPI and event services |
| FR-02 Account and access management | Cognito authorize/callback/refresh, account administration, local event-role checks | Implemented/config-dependent at the provider boundary |
| FR-03 Participant registration | Participant search/create/update, consent, emergency contacts and event registration | Implemented |
| FR-04 Queue management | Event queues, station hand-off, join/call/start/advance/complete/skip/priority | Implemented |

| Requirement | Current API evidence | Status |
|---|---|---|
| FR-05 Screening results and flags | Visual-acuity, refraction, colour-vision and eye-health preview/save operations; server-side rules and acknowledgement | Implemented for all four station types |
| FR-06 Review and referral | Review list/detail/decision and referral issue/revision/acknowledgement/document operations | Implemented |
| FR-07 Dashboard and reporting | Metrics, analytics, operations report and PDF/CSV export jobs | Implemented as source paths; no measured production result claimed |
| NFR-OFFLINE | Installable PWA, POST /api/v1/events/{eventId}/sync/screening, encrypted IndexedDB pack and durable sync ledger | Implemented for scoped four-station flow |

# 5. PostgreSQL design and limitations

PostgreSQL is the selected persistence model. Prisma provides the normal application access path, while PostgreSQL supplies foreign keys, unique constraints, indexes, JSON/JSONB fields, transactions and database triggers where migrations define them. Representative models include User, Event, Participant, EventRegistration, Station, QueueEntry, ScreeningResult, Review, Referral, AuditLog, AuthAuditLog, SyncAction and SyncActionTransition.

The design has these explicit limitations:

• A PostgreSQL server and valid DATABASE URL are prerequisites. The dated

AWS evidence records private encrypted RDS with seven-day backups and an encrypted snapshot; the repeatable local recovery test proves dump/restore integrity, not regional disaster recovery or high availability.

• The database schema is migration-owned. backend/db/init.sql is a guard

against the retired legacy schema and must not be used as a bootstrap.

• The current API performs clinical rule evaluation in the screening service;

the client repeats a bounded preliminary evaluation for offline UX. Neither is a diagnosis engine.

• Report and analytics code contains PostgreSQL-specific aggregation such as

continuous percentiles. Porting to another database would require a deliberate query and evidence review.

• The offline ledger persists safe metadata only; clinical bodies are accepted

in the authenticated request and written through the normal screening services.

• The deployed RDS instance is Single-AZ and the application uses one EC2 API

host. These choices fit the school lab but leave a documented availability ceiling.

## Stored-procedure and function evidence

backend/stored procedures.sql contains source evidence for timestamp and clinical trigger functions, queue-transfer and screening procedures, completion/category helper functions, a materialized view and a refresh procedure. The file references legacy table names such as visual acuity results and station queues, while the canonical Prisma schema uses models such as ScreeningResult and QueueEntry. It is therefore reported as a database-design artifact, not as verified deployed behavior.

Migration-backed database functions/triggers that are part of the current schema include timestamp maintenance and audit-log immutability guards; see backend/prisma/migrations/ for their SQL. A deployment claim for the standalone procedure file requires a human-run schema compatibility review and database execution evidence.

# 6. Security design

The implemented request path is:

```
HTTPS request
  → request context / security headers / rate limits
  → authentication and session checks
  → event membership, role and duty authorization
  → Zod request validation
  → controller and service transaction
  → Prisma parameterized access
  → PostgreSQL constraints and audit records
```

Evidence-backed controls include:

- Secure-cookie Cognito session flow with CSRF handling; browser code does not receive refresh tokens or CSRF values from the refresh response.
- Backend authorization checks for global account state plus event membership, role and active duty; a global role name alone is not sufficient for station writes.
- Zod validation, bounded request bodies, safe problem responses, Helmet headers, exact CORS origins, rate limits and request IDs.
- Prisma transactions, idempotency keys and request fingerprints for sensitive writes and offline sync.
- Encrypted AES-GCM IndexedDB records bound to the authenticated owner and event; pass tokens are stripped from the offline queue snapshot and expired packs are purged.
- Append-only audit and authentication logs plus immutable database triggers in the current migrations.
- Dependency and contract checks available through the repository workflows.

Deployment evidence adds a private RDS security-group boundary, encrypted storage, an RDS-owned Secrets Manager credential, TLS termination, seven-day automated backups and an encrypted snapshot. The EC2 process currently keeps a static database credential copy in root-owned /etc/vsms.env; rotation must be followed by an environment refresh. No WAF, Multi-AZ failover, regional DR or 24-hour operations team is claimed.

# 7. Testing and measured results

VSMS uses the smallest useful layered checks: unit tests for rules and security helpers, route/service integration tests for authorization and transactions, contract checks against OpenAPI, frontend component/offline tests, and a synthetic database-backed workflow for load and recovery. CI also runs dependency, secret and static-analysis checks. Test identities and data are synthetic; evidence bundles must never contain live NRICs, cookies, credentials or raw clinical payloads.

The 13 August performance run created exactly 500 synthetic participant records, registered them across two test events, exercised registration, check-in, queue reads, four-station screening sync, reporting, 500 concurrent participant pollers and ten staff queue pollers, then backed up and restored the database. All acceptance thresholds passed:

| Operation | Volume | Result |
|---|---|---|
| Registration write | 500 | 234.70 requests/s; p95 28.35 ms; 0% errors |
| Manual check-in write | 20 | p95 32.56 ms; 0% errors |

| Operation | Volume | Result |
|---|---|---|
| Screening sync | 20 batches / 500 actions | p95 343.96 ms; 0% errors |
| Participant status polling | 2,997 requests | 99.89 requests/s; p95 22.28 ms; 0% errors |
| Staff queue polling | 31 requests | p95 32.09 ms; 0% errors |
| Aggregate reporting | measured workflow | p95 33.52 ms; 0% errors |

The enforced budgets are p95 at most 250 ms for reads, p95 at most 500 ms for writes, and at most 1% errors. Peak API CPU was 259.3% of one core on a ten-logical-CPU host (about 26% normalized) with 419,424 KiB resident memory (about 1.2% of 32 GiB). These are repeatable lab measurements, not an RDS or internet capacity guarantee. The complete sanitized method and result are in docs/2026-08-13-performance-recovery.md.

The same run created a PostgreSQL custom-format dump and restored it into a fresh database. Verification compared exact row counts plus all 177 constraints and 245 index definitions. The restored counts included 500 participants, 1,000 event registrations, 520 queue entries, 500 screening results, 500 synchronization actions and 2,070 audit logs. The manual performance-recovery.yml workflow repeats the load and restore sequence in a GitHub-hosted PostgreSQL 16 service and retains only synthetic private artifacts for 30 days.

# 8. Verification record

The following commands reproduce the branch checks. Passing them proves source and isolated-database behavior; live Cognito, provider delivery and deployed role journeys still require the final acceptance replay.

```
pnpm --dir backend prisma:validate
pnpm --dir backend openapi:lint
pnpm --dir backend contracts:check
pnpm --dir backend test
pnpm --dir react-user-dashboard lint
pnpm --dir react-user-dashboard test
pnpm --dir react-user-dashboard build
pnpm check:api-collection
pnpm check:submission-package
```

pnpm package:submission creates the deterministic source-only archive. It excludes dependencies, environment files, secrets, logs, raw evidence and database dumps. The final DBSP ZIP must additionally contain the official individual report PDF, source ZIP, SQL database ZIP and signed academic integrity declaration under the filenames required by the brief.

# 9. 15-minute demo outline

The evidence-driven run sheet is in demo-outline.md. It follows a path supported by the current routes and frontend:

```
login → event/participant operations → online screening
→ preloaded offline pack → offline save → reconnect and sync
→ review/referral → aggregate dashboard/report → security boundary recap
```

The offline segment preloads an assigned pack while online, then proves cached app-shell access, encrypted local save, reconnect, idempotent synchronization and conflict handling. The final run must retain sanitized screenshots and timestamps for each acceptance-matrix row.

# 10. Reflection and limitations

The most important design lesson was that security and simple orchestration reinforce each other. Keeping HTTP mapping in controllers, domain decisions in services and persistence at the Prisma boundary made event-scoped authorization and audit behavior easier to inspect. Likewise, reusing the online screening service from offline synchronization prevented the offline path from becoming a second clinical rules engine.

Performance testing exposed two avoidable bottlenecks: Serializable transactions were used where row-level locking was sufficient, and queue polling returned completed history when clients needed only active work plus totals. Moving contention control to the relevant event or registration row and returning the smaller operational set improved throughput without adding a cache or new infrastructure. Recovery testing also showed that row counts alone are inadequate; a usable restore must preserve constraints and indexes.

Remaining risk is operational rather than hidden in the report. The AWS lab is Single-AZ, uses one EC2 application host, relies on refreshed static database credentials after rotation, and was not reachable end to end during the final repository audit. A production evolution would add Multi-AZ RDS, more than one API instance behind a load balancer, shared fail-closed rate-limit and idempotency infrastructure, automated secret retrieval, monitoring and a tested regional recovery plan. Those changes should be made only when the availability requirement justifies their cost.

# 11. Human-owned final inputs

These items intentionally remain open:

| Input | Repository state |
|---|---|
| Exact student name and identifier for the individual DBSP filename | Not recoverable from repository history; required before final ZIP naming |
| Official DBSP individual-report template | Brief says it is on BrightSpace; the template is not in this repository |
| Official declaration, name, date and signature | BrightSpace form must be supplied and signed by the student; no signature is invented |
| External AI/chat links, if required by the course | Use docs/ai-transcripts/EXTERNAL AI CHAT LINKS.md; do not invent links |
| Lecturer/course-specific report formatting or manual report template | Apply manually to this source; no unverified template is claimed |
| Production PostgreSQL SQL backup | Generate from the authorized environment; the tested custom dump is recovery evidence, not the brief's required SQL file |
| Lucidchart ERD iterations and final editable link | Mermaid sources are supplied; the required Lucidchart ownership/link must come from the team |
| Current AWS/Cognito acceptance screenshots | Renew AWS Academy access and replay the matrix with representative test accounts |

# 12. References

- backend/docs/openapi.yaml
- backend/prisma/schema.prisma
- backend/docs/offline-screening-28.md
- backend/services/SERVICES.md
- README.md
- docs/2026-08-11 aws-cloud-deployment-runbook.md
- docs/2026-08-13-performance-recovery.md
- docs/featureList.md
- diagrams/

# 13. Diagram index

| Required view | Canonical editable source | Status |
|---|---|---|
| System context | diagrams/ContextDiagram.md | Corrected to the implemented AWS and application boundary |
| Use cases | diagrams/UseCaseDiagram.md | Role and event-scope paths |
| NoSQL design | diagrams/NoSQLDesign.md | Evaluated query-first alternative, explicitly not implemented |
| Component view | diagrams/ComponentDiagram.md | React, Express, services, Prisma and workers |
| Deployment | diagrams/deploymentDiagram.md | Historical Amplify, EC2/Nginx, private RDS and Cognito topology |
| Request sequence | diagrams/SequenceDiagram.md | Online and four-station offline paths |
| Security architecture | diagrams/SecurityArchitecture.md | Browser, middleware, authorization, data and audit boundaries |
| Offline synchronization | diagrams/OfflineSynchronization.md | PWA shell, encrypted pack, outbox, retry and conflict states |
| Secure API | diagrams/SecureApiDesign.md | HTTPS through validation, transaction and audit |
| Relational ERD | diagrams/PostgreSQL ERD Design.md | Focused Prisma/PostgreSQL implementation model |